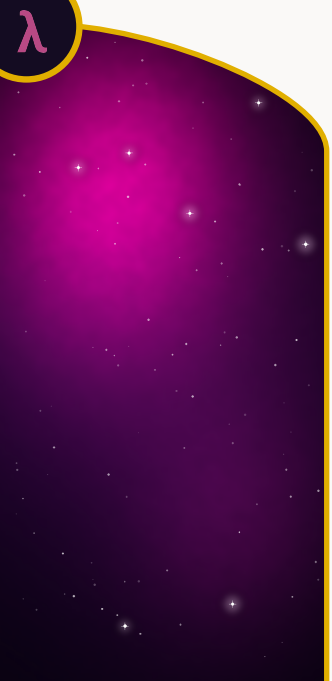


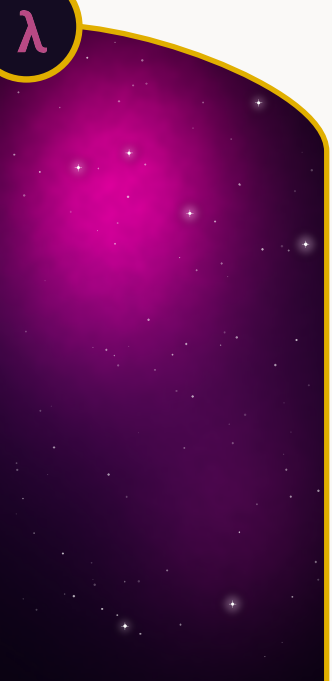
L13A: Types, Classes, and Objects

CS1101S: Programming Methodology

Boyd Anderson

11th November, 2026

Version 1.0 — Built: 2026-11-10 17:26:56 SGT



Types

- Catching bugs before they happen
- Types as sets of values
- Function, pair and linked list types
- None and Optional
- Generics
- Typing Python lists

Classes and Objects

Wrap-up

A bug that hides

```
def square(x):
    return x * x

print(square(5))          # prints 25

# many other functions and calls later
print(square("five"))
```

A bug that hides

```
def square(x):
    return x * x

print(square(5))          # prints 25

# many other functions and calls later
print(square("five"))
```



- In Python §4, the program runs and prints 25. The mistake is found only *while running*, when `"five" * "five"` is evaluated inside `square`.

A bug that hides

```
def square(x):
    return x * x

print(square(5))          # prints 25

# many other functions and calls later
print(square("five"))
```



- In Python §4, the program runs and prints 25. The mistake is found only *while running*, when `"five" * "five"` is evaluated inside `square`.
- That can happen much later in the program, or only on a path that rarely runs: the bug can hide for a long time.

The same bug, in Python §4 Typed

```python
def square(x: int) -> int:
    return x * x

print(square(5))

# many other functions and calls later
print(square("five"))
```

The same bug, in Python §4 Typed

```python
def square(x: int) -> int:
    return x * x

print(square(5))

# many other functions and calls later
print(square("five"))
```



Python §4 Typed checker

```
line 7:  square expects x to be int, but got str
```

The same bug, in Python §4 Typed

```python
def square(x: int) -> int:
    return x * x

print(square(5))

# many other functions and calls later
print(square("five"))
```



Python §4 Typed checker

```
line 7:  square expects x to be int, but got str
```

- New syntax: `x: int` (`x` is an `int`) and `-> int` (`square` returns an `int`).

The same bug, in Python §4 Typed

```python
def square(x: int) -> int:
    return x * x

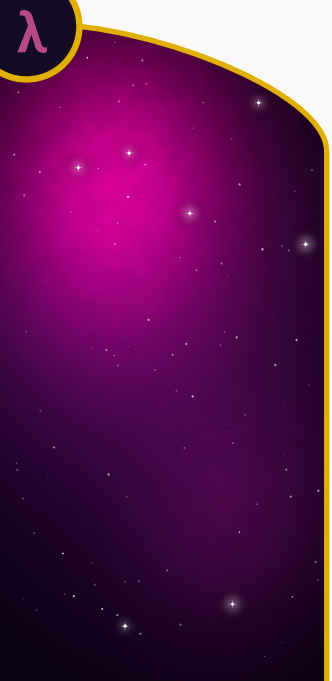
print(square(5))

# many other functions and calls later
print(square("five"))
```

Python §4 Typed checker

```
line 7:  square expects x to be int, but got str
```

- New syntax: `x: int` (`x` is an `int`) and `-> int` (`square` returns an `int`).
- Rejected *before it runs*: nothing is printed.



Python §4 Typed

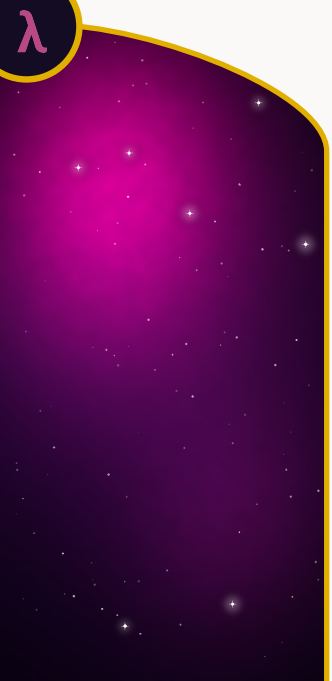
- A new language in Source Academy: Python §4 plus *type annotations*.

Python §4 Typed

- A new language in Source Academy: Python §4 plus *type annotations*.
- Programs are parsed, then type-checked, and run only if there are no type errors.

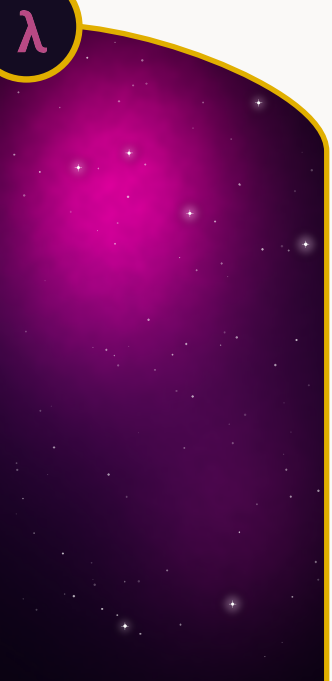
Python §4 Typed

- A new language in Source Academy: Python §4 plus *type annotations*.
- Programs are parsed, then type-checked, and run only if there are no type errors.
- At run time the annotations are ignored, so every program that passes the checker is also a Python §4 program.



Python §4 Typed

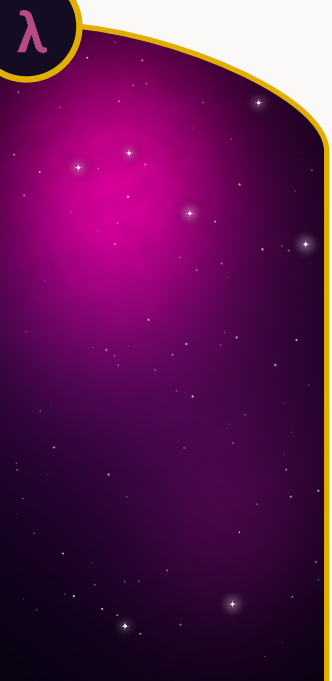
- A new language in Source Academy: Python §4 plus *type annotations*.
- Programs are parsed, then type-checked, and run only if there are no type errors.
- At run time the annotations are ignored, so every program that passes the checker is also a Python §4 program.
- Every parameter and every return type must be annotated.



The new syntax

```
def square(x: int) -> int:
```

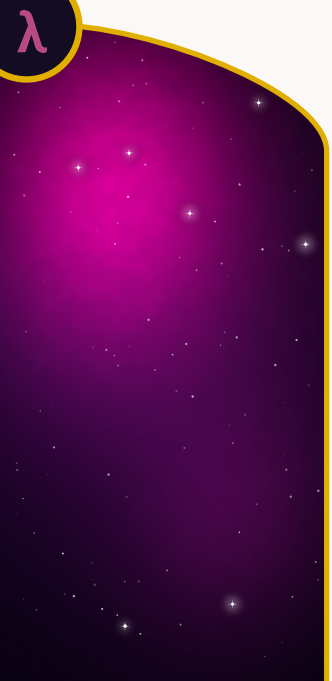
parameter and result types



The new syntax

```
def square(x: int) -> int:
```
parameter and result types

```
count: int = 0
```
the type of a name



The new syntax

```
def square(x: int) -> int:
```
parameter and result types

```
count: int = 0
```
the type of a name

```
def identity[A](x: A) -> A:
```
generic: `A` is any type

The new syntax

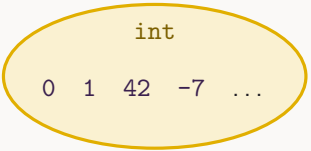
`def square(x: int) -> int:`	parameter and result types
`count: int = 0`	the type of a name
`def identity[A](x: A) -> A:`	generic: `A` is any type
`type IntFun = Callable[[int], int]`	a name for a type

The new syntax

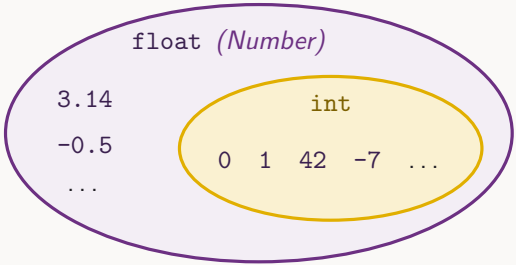
`def square(x: int) -> int:`	parameter and result types
`count: int = 0`	the type of a name
`def identity[A](x: A) -> A:`	generic: `A` is any type
`type IntFun = Callable[[int], int]`	a name for a type

That is all. Everything else is Python §4.

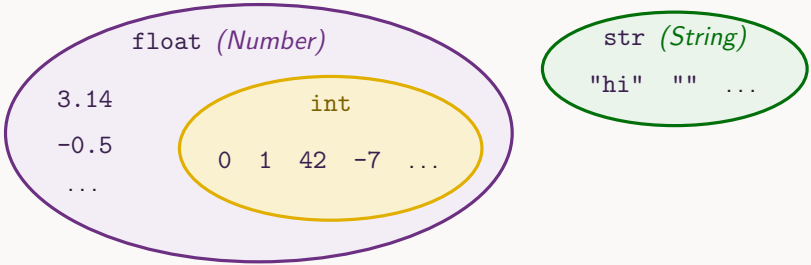
A type is a set of values



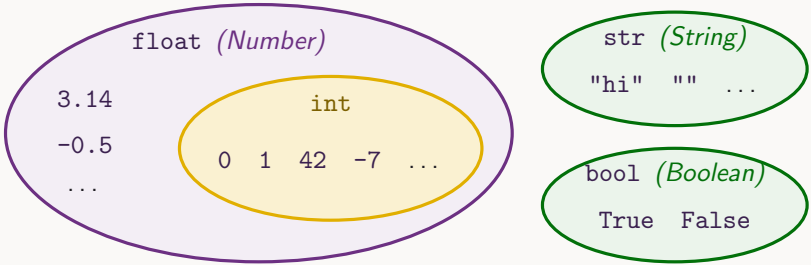
A type is a set of values



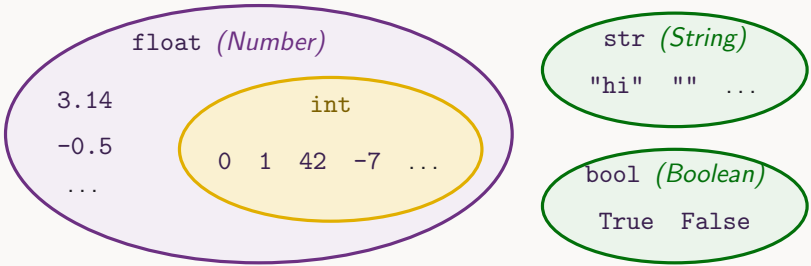
A type is a set of values



A type is a set of values

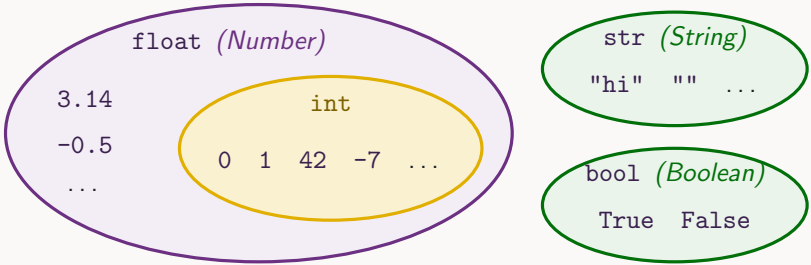


A type is a set of values

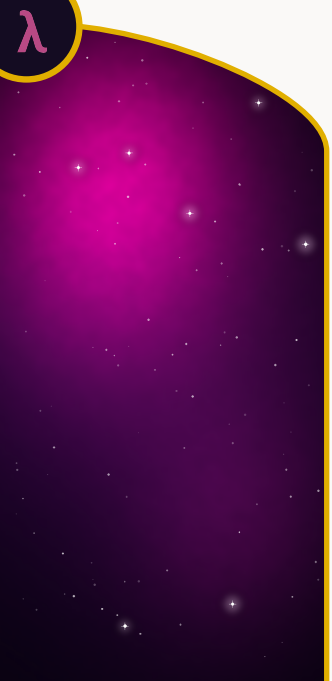


- A **type** is a set of values; it determines which operations make sense.

A type is a set of values

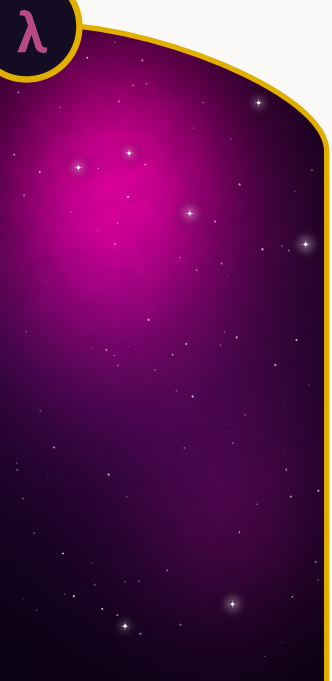


- A **type** is a set of values; it determines which operations make sense.
- Every `int` is also a `float`: the set `int` is contained in the set `float`.



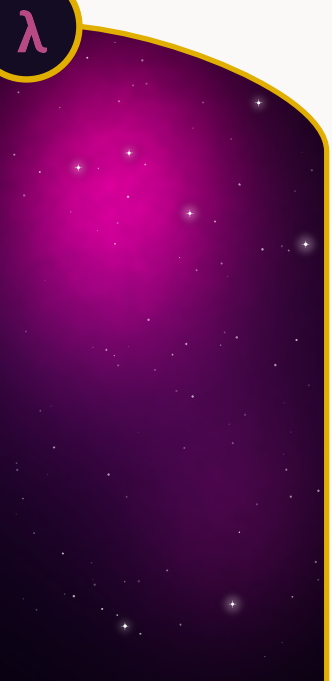
Subtypes

- If every value of type S is also a value of type T, then S is a **subtype** of T.



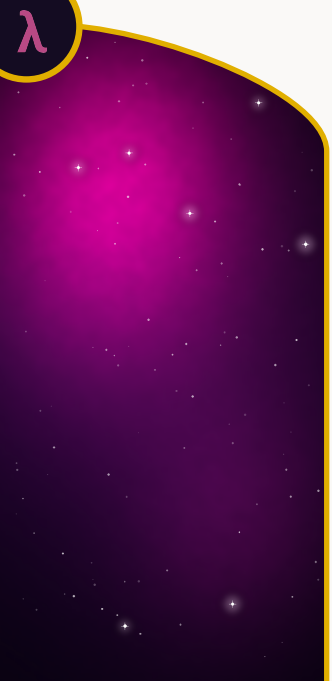
Subtypes

- If every value of type S is also a value of type T, then S is a **subtype** of T.
- `int` is a subtype of `float`.



Subtypes

- If every value of type S is also a value of type T, then S is a **subtype** of T.
- `int` is a subtype of `float`.
- A value of a subtype can be used wherever the larger type is expected.



Subtypes

- If every value of type S is also a value of type T, then S is a **subtype** of T.
- `int` is a subtype of `float`.
- A value of a subtype can be used wherever the larger type is expected.
- The type `int | None` contains every `int`, and also `None`. So `int` is a subtype of `int | None`.

Poll

```
def half(x: float) -> float:
    return x / 2

def double(n: int) -> int:
    return 2 * n
```



1. Is `half(3)` accepted?

Poll

```
def half(x: float) -> float:
    return x / 2

def double(n: int) -> int:
    return 2 * n
```



1. Is `half(3)` accepted? Yes: `3` is an `int`, and an `int` can be used where a `float` is expected.

Poll

```
def half(x: float) -> float:
    return x / 2

def double(n: int) -> int:
    return 2 * n
```



1. Is `half(3)` accepted? Yes: `3` is an `int`, and an `int` can be used where a `float` is expected.
2. Is `double(3.5)` accepted?

Poll

```
def half(x: float) -> float:
    return x / 2

def double(n: int) -> int:
    return 2 * n
```



1. Is `half(3)` accepted? Yes: `3` is an `int`, and an `int` can be used where a `float` is expected.
2. Is `double(3.5)` accepted? No: `3.5` is a `float`, and not every `float` is an `int`.

Types of functions

```
def square(x: int) -> int:
    return x * x

def greater_than_3(x: int) -> bool:
    return x > 3
```



Types of functions

```
def square(x: int) -> int:
    return x * x

def greater_than_3(x: int) -> bool:
    return x > 3
```

We write the types of these functions like this:

```
square : (int) -> int
greater_than_3 : (int) -> bool
```

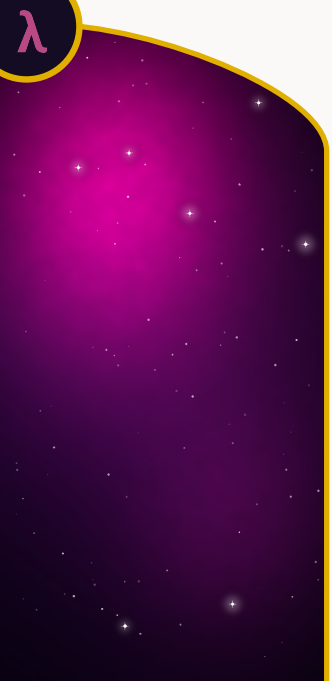
Types of functions

```
def square(x: int) -> int:
    return x * x

def greater_than_3(x: int) -> bool:
    return x > 3
```

We write the types of these functions like this:

```
square : (int) -> int
greater_than_3 : (int) -> bool
```



Writing a function type

- We write `(int) -> int`. In an annotation, the same type is written `Callable[[int], int]`: the parameter types in a list, then the return type.

Writing a function type

- We write `(int) -> int`. In an annotation, the same type is written `Callable[[int], int]`: the parameter types in a list, then the return type.
- A lambda cannot carry annotations, so its type comes from the name it is given:

```
inc: Callable[[int], int] = lambda x: x + 1
```



Writing a function type

- We write `(int) -> int`. In an annotation, the same type is written `Callable[[int], int]`: the parameter types in a list, then the return type.
- A lambda cannot carry annotations, so its type comes from the name it is given:

```
inc: Callable[[int], int] = lambda x: x + 1
```

A long type can be given a name, with a **type alias**:

```
type IntFun = Callable[[int], int]
inc: IntFun = lambda x: x + 1
```

You have seen types before

Course notation (S4, L3B)	**Python §4 Typed**
Number	`float`
Number $\to$ *Number*	`Callable[[float], float]`
(*Number*, *Number*) $\to$ *Number*	`Callable[[float, float], float]`
Number-Transformation $\to$ *Number*	`Callable[[NumberTransformation], float]`

You have seen types before

Course notation (S4, L3B)	**Python §4 Typed**
Number	`float`
Number $\to$ *Number*	`Callable[[float], float]`
(*Number*, *Number*) $\to$ *Number*	`Callable[[float, float], float]`
Number-Transformation $\to$ *Number*	`Callable[[NumberTransformation], float]`

Number-Transformation := *Number* $\to$ *Number*

is written

```
type NumberTransformation = Callable[[float], float]
```

You have seen types before

Course notation (S4, L3B)	**Python §4 Typed**
Number	`float`
Number $\to$ *Number*	`Callable[[float], float]`
(*Number*, *Number*) $\to$ *Number*	`Callable[[float, float], float]`
Number-Transformation $\to$ *Number*	`Callable[[NumberTransformation], float]`

Number-Transformation := *Number* $\to$ *Number*

is written

```
type NumberTransformation = Callable[[float], float]
```

The same types, written in Python.

Functions as arguments and results



```
def thrice(f: IntFun) -> IntFun:
    return lambda x: f(f(f(x)))

def label(x: int) -> str:
    return "big" if x > 3 else "small"

thrice(square)(2)    # 256
thrice(label)(2)
```

Functions as arguments and results

```
def thrice(f: IntFun) -> IntFun:
    return lambda x: f(f(f(x)))

def label(x: int) -> str:
    return "big" if x > 3 else "small"

thrice(square)(2)    # 256
thrice(label)(2)
```

Python §4 Typed checker

```
line 8:  thrice expects f to be IntFun, but got (x:  int) ->
str
```

Functions as arguments and results

```
def thrice(f: IntFun) -> IntFun:
    return lambda x: f(f(f(x)))

def label(x: int) -> str:
    return "big" if x > 3 else "small"

thrice(square)(2)    # 256
thrice(label)(2)
```

Python §4 Typed checker

```
line 8:  thrice expects f to be IntFun, but got (x:  int) ->
str
```

`label` accepts an `int`, but `thrice` passes each result back into `f`, so the result must be an `int` too.

The pair type

```
p: Pair[int, str] = pair(1, "one")
n: int = head(p)
s: str = tail(p)

t: str = head(p)
```



The pair type

```
p: Pair[int, str] = pair(1, "one")
n: int = head(p)
s: str = tail(p)

t: str = head(p)
```

Python §4 Typed checker

```
line 5:  Type "int" is not assignable to declared type "str"
```

The pair type

```
p: Pair[int, str] = pair(1, "one")
n: int = head(p)
s: str = tail(p)

t: str = head(p)
```

Python §4 Typed checker

```
line 5:  Type "int" is not assignable to declared type "str"
```

`Pair[H, T]` is the type of pairs whose head has type `H` and whose tail has type `T`. The checker knows that `head(p)` is an `int` and `tail(p)` a `str`.

Linked lists and streams

```
# predeclared
type LList[T] = Pair[T, LList[T]] | None
type Stream[T] = Pair[T, Callable[[], Stream[T]]] | None
```

Linked lists and streams

```
# predeclared
type LList[T] = Pair[T, LList[T]] | None
type Stream[T] = Pair[T, Callable[[], Stream[T]]] | None
```

- A linked list of `T` is either `None`, or a pair whose head is a `T` and whose tail is a linked list of `T`.

Linked lists and streams

```
# predeclared
type LList[T] = Pair[T, LList[T]] | None
type Stream[T] = Pair[T, Callable[[], Stream[T]]] | None
```

- A linked list of `T` is either `None`, or a pair whose head is a `T` and whose tail is a linked list of `T`.
- A stream of `T` is either `None`, or a pair whose head is a `T` and whose tail is a function that returns the rest of the stream.

Linked lists and streams

```
# predeclared
type LList[T] = Pair[T, LList[T]] | None
type Stream[T] = Pair[T, Callable[[], Stream[T]]] | None
```

- A linked list of `T` is either `None`, or a pair whose head is a `T` and whose tail is a linked list of `T`.
- A stream of `T` is either `None`, or a pair whose head is a `T` and whose tail is a function that returns the rest of the stream.
- Both types are recursive, with the same shape as the recursive functions we write over linked lists and streams.

The head of a linked list

```
xs = llist(1)
print(head(tail(xs)))
```



The head of a linked list

```
xs = llist(1)
print(head(tail(xs)))
```



In Python §4, this program runs, and fails while running:
`tail(xs)` is empty, so it has no head.

The head of a linked list, in Python §4 Typed

```
xs: LList[int] = llist(1, 2, 3)
print(head(xs))
```



The head of a linked list, in Python §4 Typed

```
xs: LList[int] = llist(1, 2, 3)
print(head(xs))
```



Python §4 Typed checker

```
line 2:  head expects p to be Pair[H, T], but got LList[int].
The value can be None:  test for that first, for example with
is_none(...)
```

The head of a linked list, in Python §4 Typed

```
xs: LList[int] = llist(1, 2, 3)
print(head(xs))
```



Python §4 Typed checker

```
line 2:  head expects p to be Pair[H, T], but got LList[int].
The value can be None:  test for that first, for example with
is_none(...)
```

Rejected, although `xs` is not empty: the checker does not run the program, and an `LList[int]` can be `None`.

The head of a linked list, in Python §4 Typed

```
xs: LList[int] = llist(1, 2, 3)
print(head(xs))
```



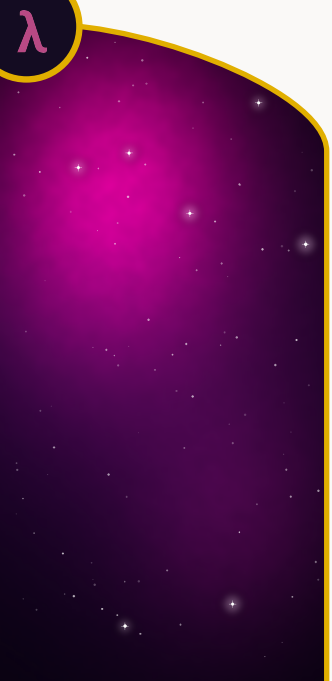
Python §4 Typed checker

```
line 2:  head expects p to be Pair[H, T], but got LList[int].
The value can be None:  test for that first, for example with
is_none(...)
```

Rejected, although `xs` is not empty: the checker does not run the program, and an `LList[int]` can be `None`.
With a test for `None` first, it is accepted:

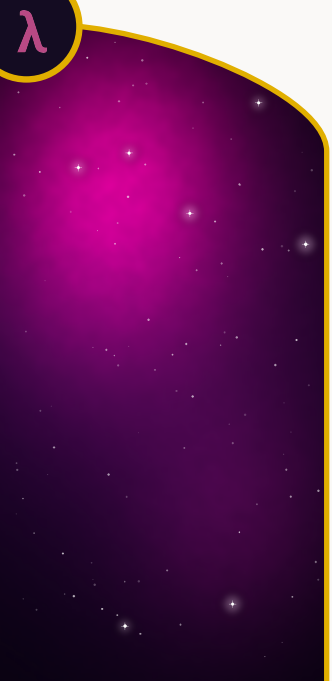
```
xs: LList[int] = llist(1, 2, 3)
print(0 if is_none(xs) else head(xs))
```





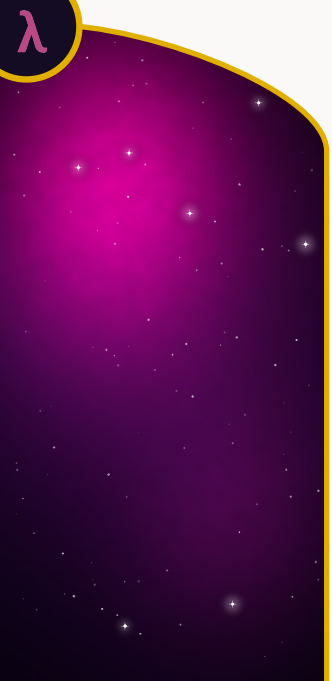
The billion-dollar mistake

- Python's `None` means a missing value. Many languages have the same idea under another name: `null` in Java, for example.



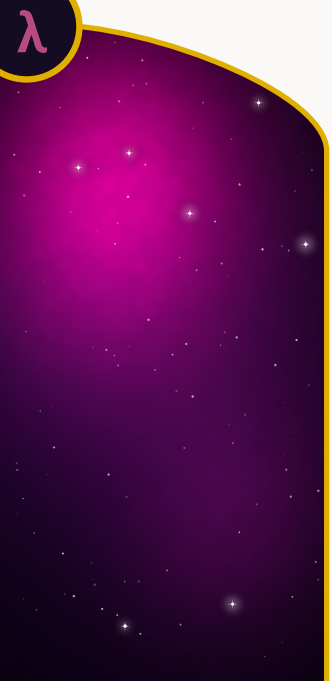
The billion-dollar mistake

- Python's `None` means a missing value. Many languages have the same idea under another name: `null` in Java, for example.
- In many of them, `null` can sneak into almost any type, and the program fails only when it is used.



The billion-dollar mistake

- Python's `None` means a missing value. Many languages have the same idea under another name: `null` in Java, for example.
- In many of them, `null` can sneak into almost any type, and the program fails only when it is used.
- Tony Hoare, who introduced `null` in 1965, later called it his "billion-dollar mistake".



The billion-dollar mistake

- Python's `None` means a missing value. Many languages have the same idea under another name: `null` in Java, for example.
- In many of them, `null` can sneak into almost any type, and the program fails only when it is used.
- Tony Hoare, who introduced `null` in 1965, later called it his "billion-dollar mistake".
- The fix is to make absence part of the type:
  - `T | None` in Python
  - `Optional<T>` in Java

The billion-dollar mistake

- Python's `None` means a missing value. Many languages have the same idea under another name: `null` in Java, for example.
- In many of them, `null` can sneak into almost any type, and the program fails only when it is used.
- Tony Hoare, who introduced `null` in 1965, later called it his "billion-dollar mistake".
- The fix is to make absence part of the type:
  - `T | None` in Python
  - `Optional<T>` in Java
- The checker then forces you to handle the missing case.

Checking for None

```
def first(xs: LList[int]) -> int | None:
    if is_none(xs):
        return None          # here, xs is None
    else:
        return head(xs)      # here, xs is a pair
```

Checking for None

```
def first(xs: LList[int]) -> int | None:
    if is_none(xs):
        return None          # here, xs is None
    else:
        return head(xs)      # here, xs is a pair
```

After the test `is_none(xs)`, the checker knows which case each branch is in, so `head(xs)` is allowed. The result type `int | None` says that `first` may return `None`.

Typing functions we know

```
def sum_llist(xs: LList[int]) -> int:
    return (0 if is_none(xs)
            else head(xs) + sum_llist(tail(xs)))
```



Typing functions we know

```
def sum_llist(xs: LList[int]) -> int:
    return (0 if is_none(xs)
            else head(xs) + sum_llist(tail(xs)))
```



What about functions that work for linked lists of *any* element type?

```
length : (LList[A]) -> int
map    : (Callable[[A], B], LList[A]) -> LList[B]
```

Typing functions we know

```
def sum_llist(xs: LList[int]) -> int:
    return (0 if is_none(xs)
            else head(xs) + sum_llist(tail(xs)))
```



What about functions that work for linked lists of *any* element type?

```
length : (LList[A]) -> int
map    : (Callable[[A], B], LList[A]) -> LList[B]
```

Read `map`'s type aloud: given a function from `A` to `B` and a linked list of `A`s, it returns a linked list of `B`s.

Generic functions

With types, would we need one `identity` for each type?

```
def identity_int(x: int) -> int:
    return x

def identity_str(x: str) -> str:
    return x
```



Generic functions

With types, would we need one `identity` for each type?

```
def identity_int(x: int) -> int:
    return x

def identity_str(x: str) -> str:
    return x
```



No: a **generic** function works for every type.

```
def identity[A](x: A) -> A:
    return x

n: int = identity(5)        # here, A is int
s: str = identity("hi")     # here, A is str
```



Generic functions

With types, would we need one `identity` for each type?

```
def identity_int(x: int) -> int:
    return x

def identity_str(x: str) -> str:
    return x
```

No: a **generic** function works for every type.

```
def identity[A](x: A) -> A:
    return x

n: int = identity(5)        # here, A is int
s: str = identity("hi")     # here, A is str
```

`[A]` says that `A` stands for a type. At each call, the checker replaces `A` by the type of the argument.

Generic linked list functions



```
def length[A](xs: LList[A]) -> int:
    return 0 if is_none(xs) else 1 + length(tail(xs))

def map[A, B](f: Callable[[A], B],
              xs: LList[A]) -> LList[B]:
    return (None if is_none(xs)
            else pair(f(head(xs)), map(f, tail(xs))))

map(lambda x: x * 2, llist(1, 2, 3))    # LList[int]
map(lambda x: x > 1, llist(1, 2, 3))    # LList[bool]
```

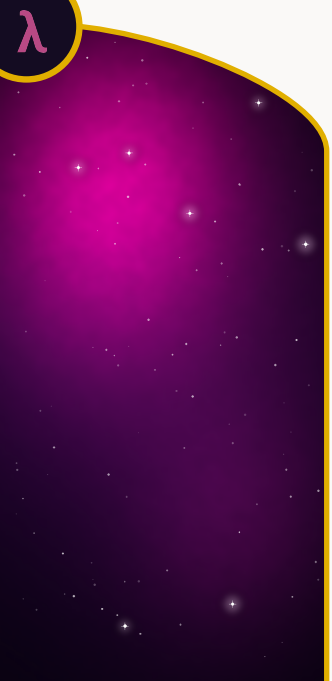
Generic linked list functions

```
def length[A](xs: LList[A]) -> int:
    return 0 if is_none(xs) else 1 + length(tail(xs))

def map[A, B](f: Callable[[A], B],
              xs: LList[A]) -> LList[B]:
    return (None if is_none(xs)
            else pair(f(head(xs)), map(f, tail(xs))))

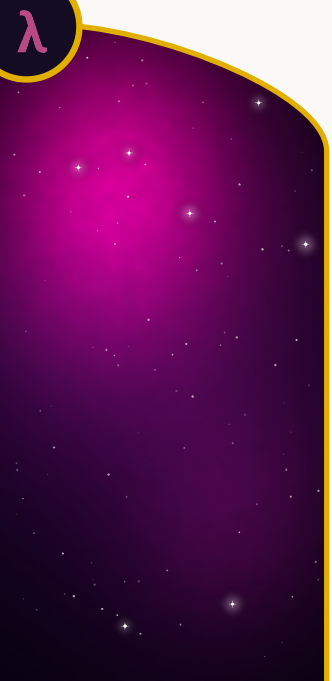
map(lambda x: x * 2, llist(1, 2, 3))    # LList[int]
map(lambda x: x > 1, llist(1, 2, 3))    # LList[bool]
```

The test `is_none(xs)` is what allows `head(xs)` and `tail(xs)` in the `else` branch: without it, the checker rejects them, because `xs` can be `None`.



Not everything can be generic

- Can `merge_sort` have type `(LList[A]) -> LList[A]`?



Not everything can be generic

- Can `merge_sort` have type `(LList[A]) -> LList[A]`?
- No: it compares elements with `<=`, which is not available on every type `A`.

Not everything can be generic

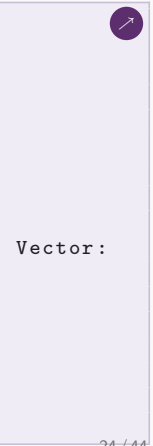
- Can `merge_sort` have type `(LList[A]) -> LList[A]`?
- No: it compares elements with `<=`, which is not available on every type `A`.
- One honest option is to fix the element type: `(LList[int]) -> LList[int]`.

Not everything can be generic

- Can `merge_sort` have type `(LList[A]) -> LList[A]`?
- No: it compares elements with `<=`, which is not available on every type `A`.
- One honest option is to fix the element type: `(LList[int]) -> LList[int]`.
- Another is to pass the comparison as an argument. In Java, that is a `Comparator`.

Typing Python lists: matrices

`list[T]` is the type of Python lists whose elements have type `T`.

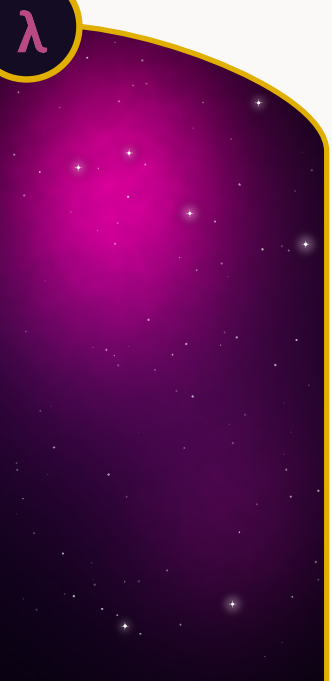


```
type Vector = list[int]
type Matrix = list[list[int]]

def dot_product(v: Vector, w: Vector) -> int:
    total = 0
    for i in range(len(v)):
        total = total + v[i] * w[i]
    return total

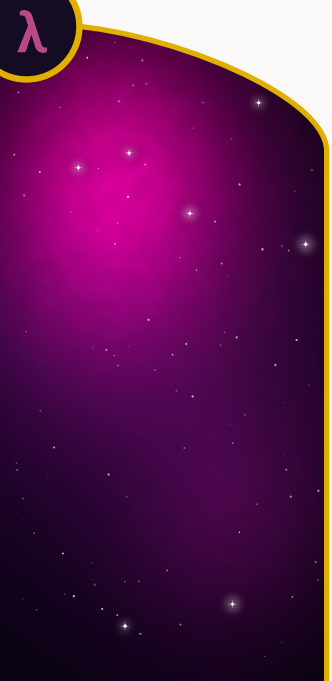
def matrix_times_vector(m: Matrix, v: Vector) -> Vector:
    result: Vector = [0] * len(m)
    for i in range(len(m)):
        result[i] = dot_product(m[i], v)
    return result

a: Matrix = [[1, 2, 3], [4, 5, 6]]
matrix_times_vector(a, [1, 0, 2])   # [7, 16]
```



What the checker gives, and what it costs

- Python §4 finds errors while the program runs, and only on the paths that actually run.



What the checker gives, and what it costs

- Python §4 finds errors while the program runs, and only on the paths that actually run.
- Python §4 Typed finds errors before the program runs, on *every* path.

What the checker gives, and what it costs

- Python §4 finds errors while the program runs, and only on the paths that actually run.
- Python §4 Typed finds errors before the program runs, on *every* path.
- The price: some programs that would have worked are rejected.

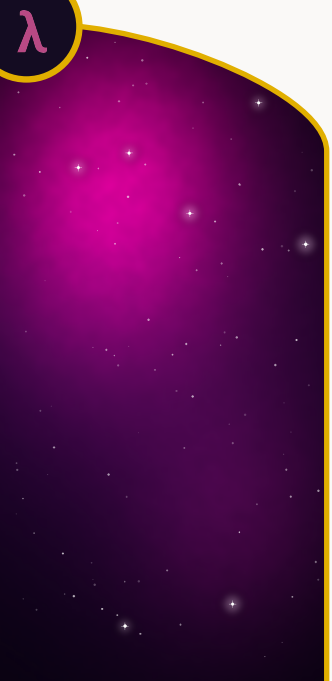
What the checker gives, and what it costs

- Python §4 finds errors while the program runs, and only on the paths that actually run.
- Python §4 Typed finds errors before the program runs, on *every* path.
- The price: some programs that would have worked are rejected.



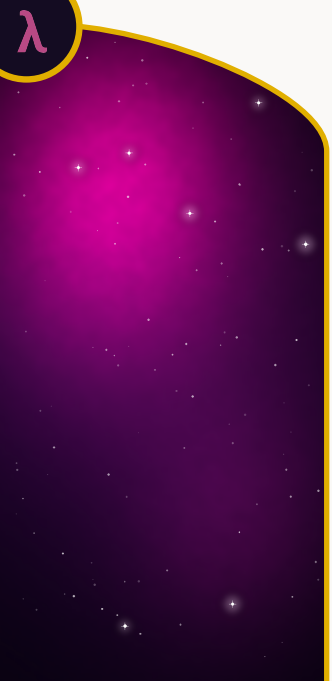
```
def describe(n: int) -> str:
    if n > 0:
        return "positive"
    else:
        return n

describe(5)     # would run fine, but is rejected
```



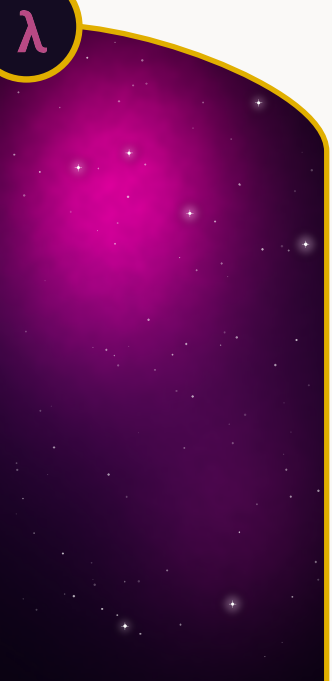
What does a type system guarantee?

- A program that passes the checker cannot go wrong in certain ways: no `"five" * "five"`, no `head` of `None`, no wrong number of arguments.



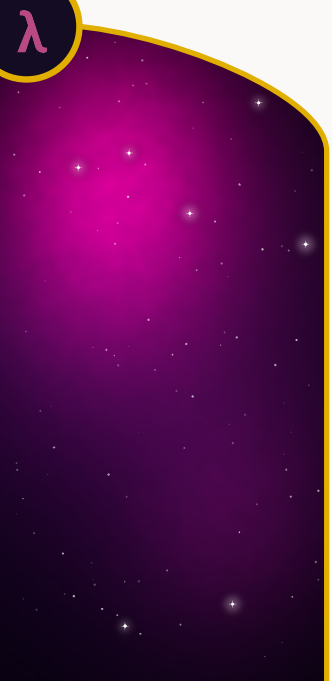
What does a type system guarantee?

- A program that passes the checker cannot go wrong in certain ways: no `"five" * "five"`, no `head` of `None`, no wrong number of arguments.
- Types do *not* guarantee that a program is correct: a sort in the wrong order, or an off-by-one error, still passes the checker.



Break

Let's take 5



Types

Classes and Objects
- Objects from closures
- Classes
- Classes are types

Wrap-up

Revisiting our bank account



```
def make_withdraw(balance):
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    return withdraw

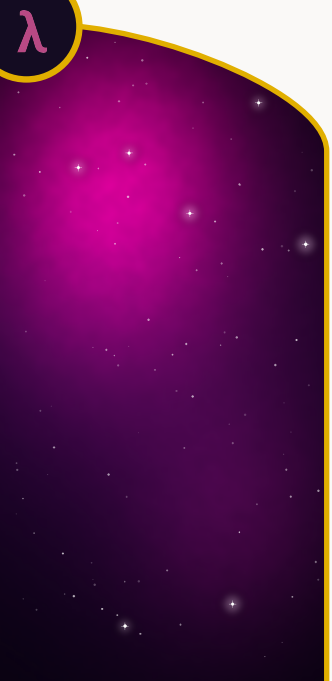
w = make_withdraw(100)
print(w(40))
print(w(70))
```

Revisiting our bank account

```
def make_withdraw(balance):
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    return withdraw

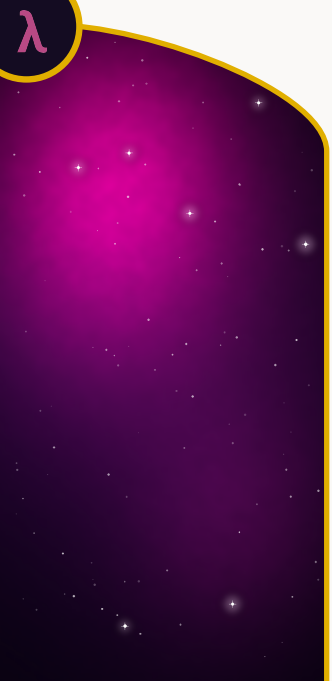
w = make_withdraw(100)
print(w(40))
print(w(70))
```

From Lecture L9A: `make_withdraw` returns a function with its own `balance`.



What if we also want to deposit?

- `make_withdraw` returns just one function, `withdraw`.



What if we also want to deposit?

- `make_withdraw` returns just one function, `withdraw`.
- An account needs several operations, such as `withdraw` and `deposit`, that share one `balance`.

What if we also want to deposit?

- `make_withdraw` returns just one function, `withdraw`.
- An account needs several operations, such as `withdraw` and `deposit`, that share one `balance`.
- Idea: return a single function that selects an operation by its name. Such a function is called a **dispatch** function.

A bank account with withdraw and deposit

```
def make_account(balance):
    def withdraw(amount):
        nonlocal balance
        if balance >= amount:
            balance = balance - amount
            return balance
        else:
            return "Insufficient funds"
    def deposit(amount):
        nonlocal balance
        balance = balance + amount
        return balance
    def dispatch(m):
        if m == "withdraw":
            return withdraw
        elif m == "deposit":
            return deposit

    return dispatch
```

Using the bank account

```
acc = make_account(100)

print(acc("withdraw")(50))
print(acc("withdraw")(60))
print(acc("deposit")(40))
```

Using the bank account

```
acc = make_account(100)

print(acc("withdraw")(50))
print(acc("withdraw")(60))
print(acc("deposit")(40))
```



- `make_account` combines data (`balance`) and the functions that work on it.

Using the bank account

```
acc = make_account(100)

print(acc("withdraw")(50))
print(acc("withdraw")(60))
print(acc("deposit")(40))
```

- `make_account` combines data (`balance`) and the functions that work on it.
- The `dispatch` function selects an operation by its name.

A circle, built the same way

```
def make_circle(radius, x, y):
    def area():
        return math_pi * radius * radius

    def contains(px, py):
        dx = px - x
        dy = py - y
        return math_sqrt(dx * dx + dy * dy) <= radius

    def dispatch(message):
        if message == "area":
            return area
        elif message == "contains":
            return contains

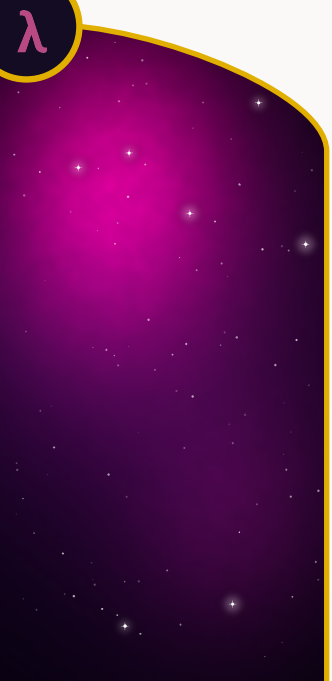
    return dispatch
```



Using the circle

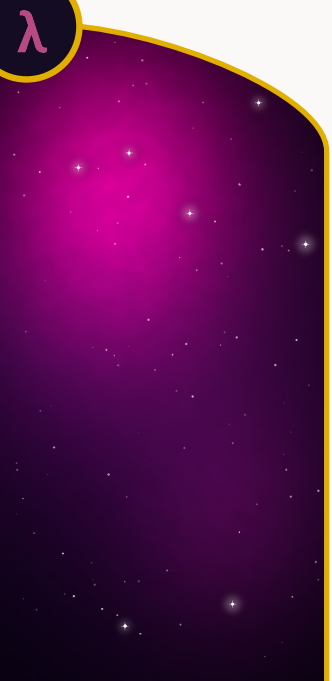
```
c = make_circle(10, 1, 1)
print(c("area")())
print(c("contains")(3, 3))
print(c("contains")(10, 10))
```





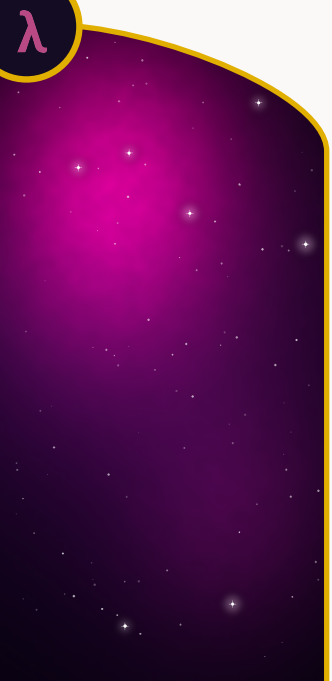
Objects

- What we built is an **object**: some state, together with the operations on it.



Objects

- What we built is an **object**: some state, together with the operations on it.
- Each call of `make_circle` creates a new, separate object with its own state.

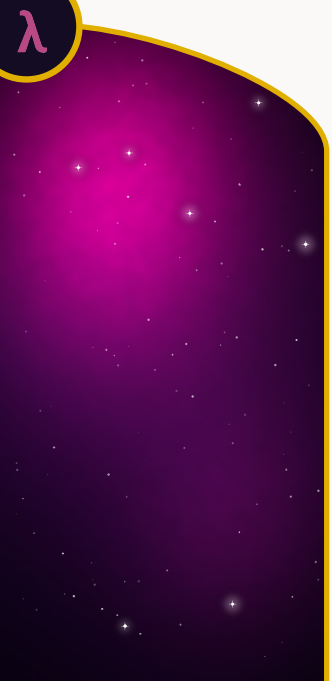


Objects

- What we built is an **object**: some state, together with the operations on it.
- Each call of `make_circle` creates a new, separate object with its own state.
- We send an object a *message* to select an operation.

Objects

- What we built is an **object**: some state, together with the operations on it.
- Each call of `make_circle` creates a new, separate object with its own state.
- We send an object a *message* to select an operation.
- This works, but every object needs its own hand-written `dispatch`, and a misspelt message is found only when the program runs.



Classes: blueprints for objects

- A **class** is a blueprint: it defines the properties (attributes) and the behaviours (methods) of its objects.

Classes: blueprints for objects

- A **class** is a blueprint: it defines the properties (attributes) and the behaviours (methods) of its objects.
- An object built from a class is an **instance** of it, with its own data.

Classes: blueprints for objects

- A **class** is a blueprint: it defines the properties (attributes) and the behaviours (methods) of its objects.
- An object built from a class is an **instance** of it, with its own data.
- Calling the class creates an instance, and `__init__` sets up its state.

Classes: blueprints for objects

- A **class** is a blueprint: it defines the properties (attributes) and the behaviours (methods) of its objects.
- An object built from a class is an **instance** of it, with its own data.
- Calling the class creates an instance, and `__init__` sets up its state.
- `c.radius` and `c.area()` select a property or method of `c`. No `dispatch` needed.

Classes: blueprints for objects

- A **class** is a blueprint: it defines the properties (attributes) and the behaviours (methods) of its objects.
- An object built from a class is an **instance** of it, with its own data.
- Calling the class creates an instance, and `__init__` sets up its state.
- `c.radius` and `c.area()` select a property or method of `c`. No `dispatch` needed.
- Python §4 has no classes, but full Python and Java do.

A Python class



```
from math import pi, sqrt

class Circle:
    def __init__(self, radius, x, y):
        self.radius = radius
        self.x = x
        self.y = y

    def area(self):
        return pi * self.radius * self.radius

    def contains(self, px, py):
        dx = px - self.x
        dy = py - self.y
        return sqrt(dx * dx + dy * dy) <= self.radius
```

Using the class

```
c = Circle(10, 1, 1)      # calls __init__
c.area()                  # 314.1592653589793
c.contains(3, 3)          # True
c.radius                  # 10
```



Using the class

```
c = Circle(10, 1, 1)      # calls __init__
c.area()                  # 314.1592653589793
c.contains(3, 3)          # True
c.radius                  # 10
```

Compare `c.area()` with `c("area")()`: the class does the dispatching for us.

The bank account as a class

```python
class Account:
    def __init__(self, balance: int) -> None:
        self.balance = balance
    def withdraw(self, amount: int) -> int | str:
        if self.balance >= amount:
            self.balance = self.balance - amount
            return self.balance
        else:
            return "Insufficient funds"
    def deposit(self, amount: int) -> int:
        self.balance = self.balance + amount
        return self.balance

acc = Account(100)
acc.withdraw(50)     # 50
acc.deposit(40)      # 90
```

The bank account as a class

```
class Account:
    def __init__(self, balance: int) -> None:
        self.balance = balance
    def withdraw(self, amount: int) -> int | str:
        if self.balance >= amount:
            self.balance = self.balance - amount
            return self.balance
        else:
            return "Insufficient funds"
    def deposit(self, amount: int) -> int:
        self.balance = self.balance + amount
        return self.balance

acc = Account(100)
acc.withdraw(50)     # 50
acc.deposit(40)      # 90
```

No `dispatch` needed, and the class *is* a type.

A class is a type

```
def total_area(cs: list[Circle]) -> float:
    total = 0.0
    for c in cs:
        total = total + c.area()
    return total

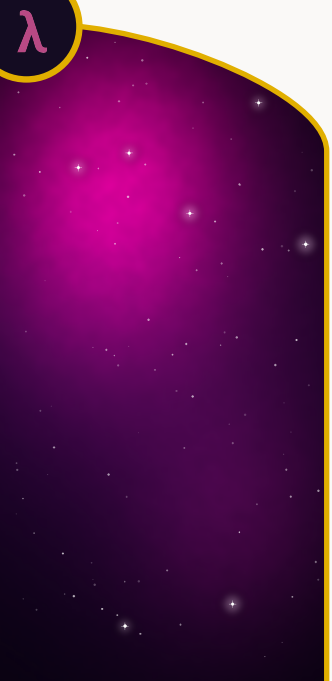
total_area([Circle(1, 0, 0), Circle(2, 5, 5)])
```

A class is a type

```
def total_area(cs: list[Circle]) -> float:
    total = 0.0
    for c in cs:
        total = total + c.area()
    return total

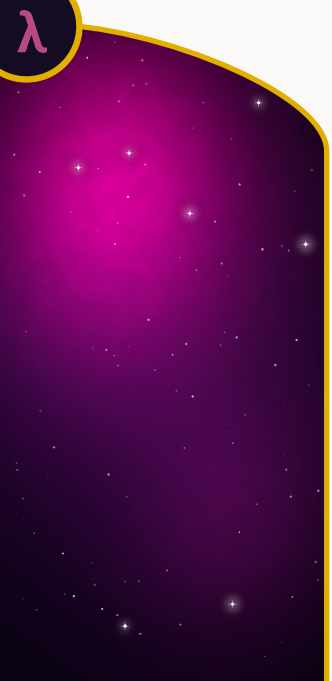
total_area([Circle(1, 0, 0), Circle(2, 5, 5)])
```

The class `Circle` from before is the set of all `Circle` objects: a new type we can use in annotations, such as `list[Circle]`.



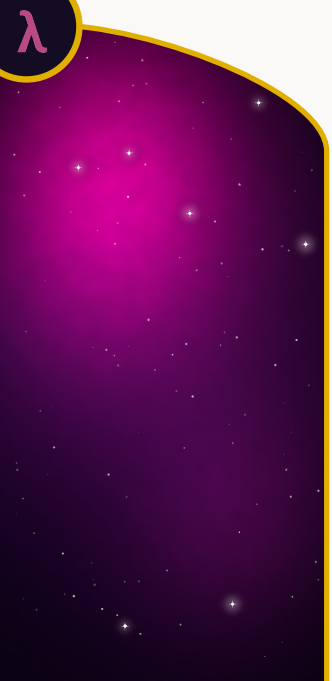
Next semester: CS2030S

- Classes and objects, in Java



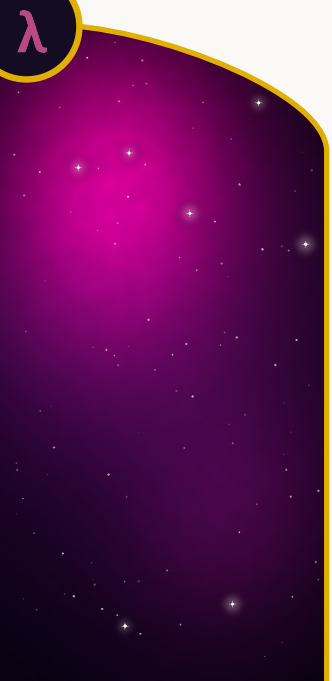
Next semester: CS2030S

- Classes and objects, in Java
- Subtyping and inheritance



Next semester: CS2030S

- Classes and objects, in Java
- Subtyping and inheritance
- Generics, and when a `List<Integer>` can be used as a `List<Number>` (it cannot: *variance*)



Next semester: CS2030S

- Classes and objects, in Java
- Subtyping and inheritance
- Generics, and when a `List<Integer>` can be used as a `List<Number>` (it cannot: *variance*)
- `Optional<T>` for missing values

Next semester: CS2030S

- Classes and objects, in Java
- Subtyping and inheritance
- Generics, and when a `List<Integer>` can be used as a `List<Number>` (it cannot: *variance*)
- `Optional<T>` for missing values
- Functions as values again: functional interfaces such as `Function<A, B>`

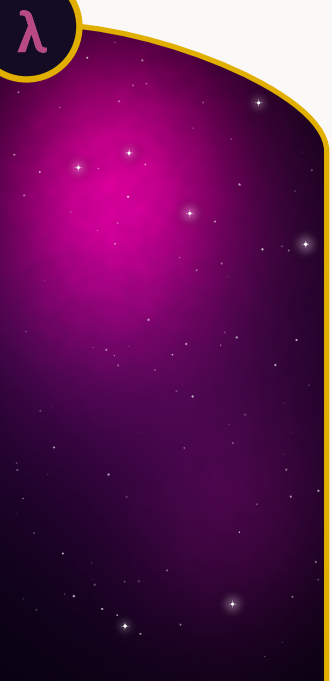
Classes in Java

```
class Circle {
    private double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    double area() {
        return Math.PI * radius * radius;
    }
}

Circle c = new Circle(1.0);
c.area();
```



Classes in Java

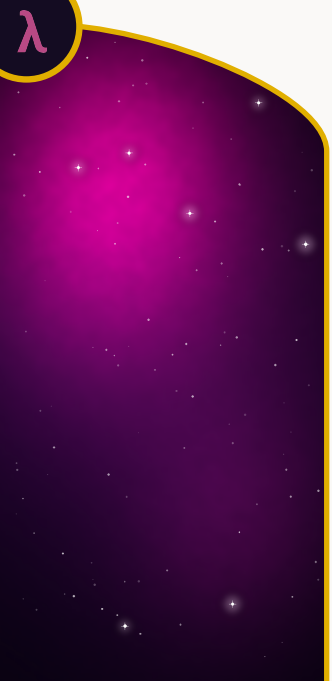
```
class Circle {
    private double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    double area() {
        return Math.PI * radius * radius;
    }
}

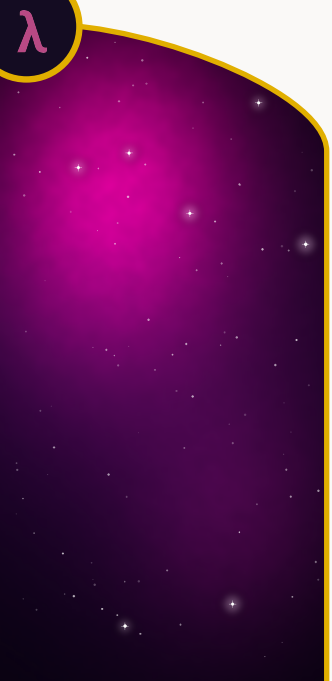
Circle c = new Circle(1.0);
c.area();
```

The same idea as the Python class `Circle`. Java declares the type of every field, parameter and result, and creates an instance with `new`.



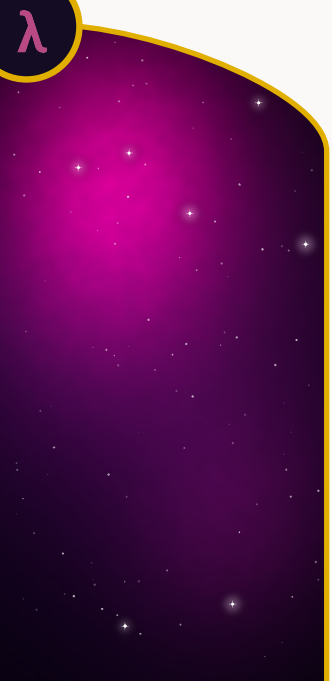
Lecture summary

- A type is a set of values; a subtype is a subset.



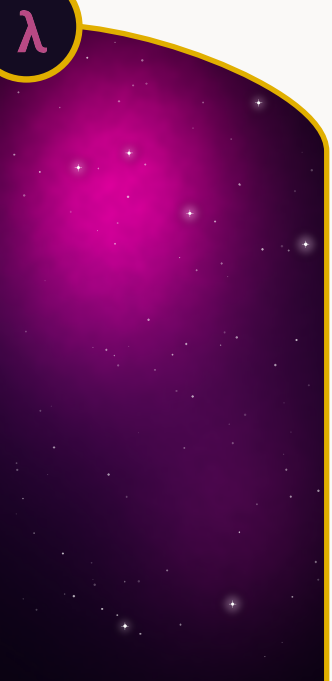
Lecture summary

- A type is a set of values; a subtype is a subset.
- A type checker finds errors before the program runs, at the price of rejecting some programs that would have worked.



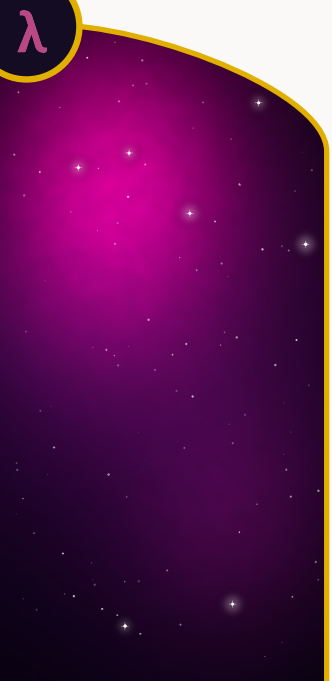
Lecture summary

- A type is a set of values; a subtype is a subset.
- A type checker finds errors before the program runs, at the price of rejecting some programs that would have worked.
- Generic functions work for any element type, and `T | None` makes absence visible.



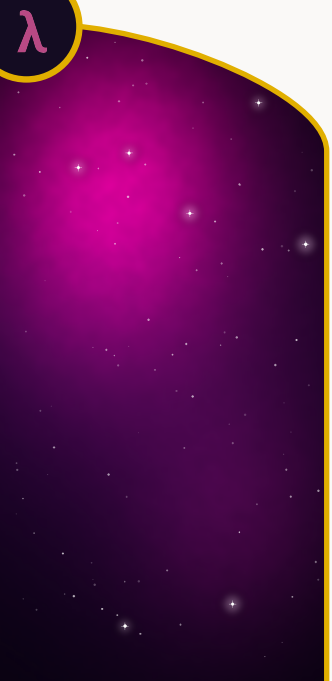
Lecture summary

- A type is a set of values; a subtype is a subset.
- A type checker finds errors before the program runs, at the price of rejecting some programs that would have worked.
- Generic functions work for any element type, and `T | None` makes absence visible.
- Objects combine state and operations; classes are blueprints for objects, and each class is a type.



Lecture summary

- A type is a set of values; a subtype is a subset.
- A type checker finds errors before the program runs, at the price of rejecting some programs that would have worked.
- Generic functions work for any element type, and `T | None` makes absence visible.
- Objects combine state and operations; classes are blueprints for objects, and each class is a type.
- Try it yourself: Python §4 Typed in Reflection R13.



Check-In time!

Class dismissed! (Right after the check-in.)

Look for Check-In I13A in the Source Academy!